Nombre: Arleth García Martínez **Nombre**: Ángel Mauricio Castruita Márquez **Nombre**: Carlos Patricio Lozano Treviño	**Matrícula**: 2145524 **Matrícula:** 2146281 **Matrícula:** 2147049
Nombre del curso: Seguridad Informática y Criptografía	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Módulo Netacad: 3	**Actividad**: Diseño arquitectural del sistema con IA (Unidad 3)
Fecha: 14/10/2026	
Bibliografia (Formato APA) • Archivo "Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx" proporcionado en la plataforma TEAMS • Archivo "10_FarmaExpress.docx" proporcionado en la plataforma TEAMS	

Objetivo:

Convertir el análisis de negocio de la Unidad 2 de FarmaExpress (BRD, historias de usuario, BPMN y glosario) en una propuesta de arquitectura de software que separe dominio, aplicación e infraestructura. Se presenta la selección del patrón de lógica de negocio, el mapa de dominio (DDD), la Service Layer, Repository y Unit of Work, CQRS básico, la revisión SOLID, los diagramas UML (clases y secuencia) y los ADR, con trazabilidad hacia la U2.

Repositorio de GitHub:

https://github.com/CrlosLzno/farmaexpress-arquitectura/invitations

Proceso principal: venta en mostrador con validación de receta, existencia, lote y caducidad, y disparo del resurtido.

Procedimiento:

1. Casos de uso seleccionados

Se eligieron cinco casos de uso que aparecen en las historias de usuario y en el BPMN (ninguno es corrección de alcance). Incluyen un flujo normal y flujos con rechazo o excepción.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado	Origen U2
CU-01	Registrar venta de producto sin restricciones (flujo normal)	Cajero	Existencia, lote vendible, descuento, ticket	Existencia descontada y ticket	US-01
CU-02	Vender medicamento controlado (rechazo y retención)	Cajero, responsable de farmacia	Receta completa y vigente (≤ 30 días); límite 10 piezas / $3,000 requiere autorización	Venta cobrada, rechazada o retenida	US-02, 03, 04 · BN-01, BN-10
CU-03	Ofrecer producto sustituto	Cajero, Responsable	Sustituto equivalente; en controlados exige autorización	Sustituto en el ticket	US-05, 06
CU-04	Bloquear caducados/retirados y registrar retiro	Responsable de farmacia	Retiro bloquea todas las sucursales;	Lote bloqueado	US-07, 08 ·

	de lote (excepción)		notifica a Almacén	globalmente	BN-02, BN-06
CU-05	Generar alerta de resurtido y orden de compra	Sistema, Compras	Existencia < punto de reorden; una orden por proveedor	Alerta y orden de compra	US-12, 13 · BN-04

2. Selección del patrón de lógica de negocio

Criterio del caso	**Transaction Script**	**Table Module**	**Domain Model**	**Evidencia U2**
Complejidad de reglas	Muchos "if" anidados repetidos en varios scripts	Reglas por tabla; las que cruzan tablas no tienen dueño	Cada regla vive en el concepto que la protege	10 necesidades y 18 historias; 5 gateways en el BPMN
Cambios esperados	Cambiar un umbral toca varios scripts	Cambios por tabla, no por regla	Cambio localizado (p. ej. límites, política de lote)	Supuestos a validar del BRD; vigencia y límites sólo en US-03/04
Interacción entre conceptos	Coordinación manual dispersa	Interacción fuera de	Objetos que colaboran con	BPMN con 4 participantes;

		los módulos	lenguaje del negocio	glosario de 28 términos
Estados e invariantes	Se validan a mano en cada script	Estado = columna, nada lo protege	La invariante se protege dentro del objeto	Restricción BRD: ningún controlado sin receta validada
Reportes y consultas	Muy adecuado	Adecuado	Costoso para reportes	US-18, US-09, US-15, BN-09

Decisión: Domain Model como patrón principal para venta, inventario por lote, retiro de lote y resurtido; Transaction Script únicamente en consultas y reportes (Query Handlers); Table Module descartado. Problema concreto que resuelve: las garantías regulatorias (receta válida, lote retirado bloqueado en todas las sucursales, autorización por rol) son invariantes que cruzan varios conceptos y cambian con el tiempo; en un script pueden olvidarse, en un objeto de dominio no pueden saltarse.

Costo aceptado: más clases e indirección (Repository, Unit of Work, eventos) y mayor curva de aprendizaje.

3. Mapa de dominio (DDD)

El mapa nace del lenguaje y las reglas de la U2, no de las tablas de datos. Las referencias entre Aggregates se hacen sólo por identidad (líneas punteadas).

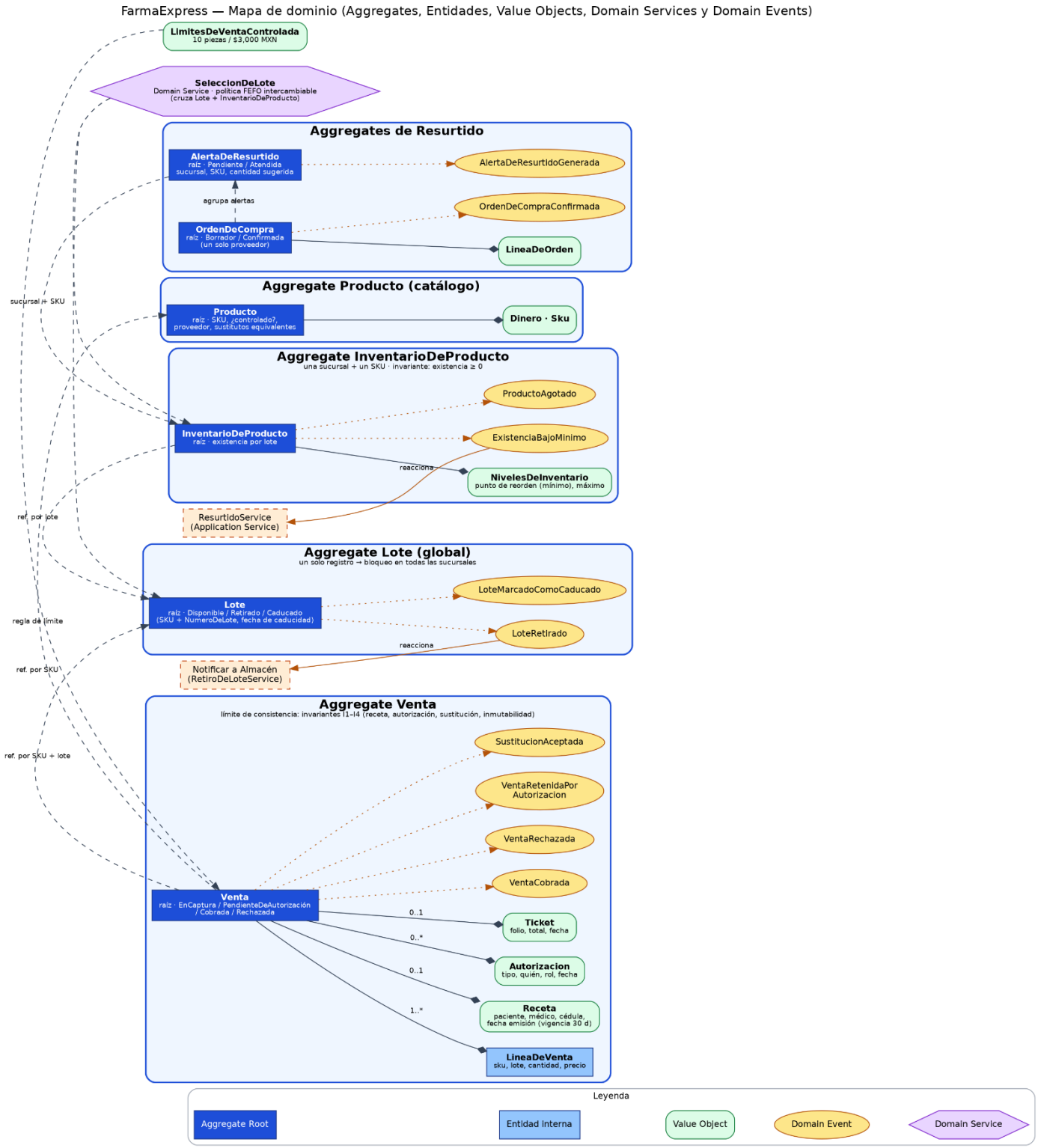


Ilustración 1 Mapa de dominio de FarmaExpress: Aggregates, entidades, Value Objects, Domain Service y Domain Events.

4. Fichas de Aggregates

Aggregate (raíz)	**Responsabilidad**	**Invariantes**	**Eventos**
Venta	Capturar productos y sustitutos, recibir receta y autorizaciones, cobrar o rechazar. Contiene LineaDeVenta, Receta, Autorizacion y Ticket.	I1 controlado ⇒ receta completa y vigente · I2 exceder límite ⇒ autorización del Responsable · I3 sustituir controlado ⇒ autorización · I4 venta cobrada/rechazada no se modifica	VentaCobrada, VentaRechazada, VentaRetenidaPorAutorizacion, SustitucionAceptada
Lote (global)	Saber si es vendible hoy, marcarse caducado, retirarse con autorización.	Retirado/Caducado no es vendible · retirar exige autorización del Responsable · idempotente	LoteRetirado, LoteMarcadoComoCaducado
InventarioDeProducto (sucursal + SKU)	Existencia por lote y niveles (mínimo, máximo); descontar, recibir, cantidad a resurtir.	Existencia nunca negativa · el evento de mínimo sólo al cruzar el punto de reorden	ExistenciaBajoMinimo, ProductoAgotado
Producto	Catálogo: SKU, precio, ¿controlado?, proveedor, sustitutos.	Un sustituto debe ser equivalente	—

AlertaDeResurtido / OrdenDeCompra	Alerta Pendiente → Atendida; orden Borrador → Confirmada.	Una orden = un proveedor y no se confirma vacía	AlertaDeResurtidoGenerada, OrdenDeCompraConfirmada

Domain Service: SeleccionDeLote (cruza Lote e InventarioDeProducto; política FEFO intercambiable, supuesto por validar). Value Objects: Sku, NumeroDeLote, Dinero, Receta, Autorizacion, Ticket, LimitesDeVentaControlada, NivelesDeInventario.

Decisiones dudosas o descartadas

- Receta como Value Object (no Entidad): U2 no le da ciclo de vida propio ni la reutiliza entre ventas. Cambiaría a Entidad si se permite surtido parcial.
- Existencia dentro del Lote: descartado, porque retirar un lote tendría que modificar N registros por sucursal y BN-06 exige bloqueo inmediato.
- Sucursal, Cliente/Paciente y Ticket como Aggregates: descartados; sin comportamiento propio en el alcance.

5. Service Layer

Cada método representa un caso de uso completo y coordina (carga Aggregates, invoca el dominio, pide persistencia, define la transacción) sin contener reglas del negocio.

Servicio	Método (caso de uso)	Entrada	Colaboradores	Salida	Reglas que delega
VentaMostradorService	agregar_producto (CU-01, 03)	AgregarProductoALaVenta	UoW, SeleccionDeLote, Reloj	Resultado / sustitutos	Lote vendible y existencia → SeleccionDeLote, Lote

				sugeridos	
	registrar_receta (CU-02)	RegistrarRecetaEnVenta	UoW	Resultado	Estructura de receta → Receta
	confirmar_cobro (CU-01, 02)	ConfirmarCobro	UoW, Reloj, Límites	Folio y total / rechazo / retenida	Receta, vigencia, límite → Venta; descuento → InventarioDeProducto
	aceptar_sustituto (CU-03)	AceptarSustituto	UoW, SeleccionDeLote	Resultado	Equivalencia y autorización → Venta, Autorizacion
AutorizacionDeVentaService	otorgar (CU-02)	OtorgarAutorizacionDeVenta	UoW, Reloj	Estado de la venta	Quién puede autorizar → Autorizacion

RetiroDeLoteService	registrar_retiro (CU-04)	RegistrarRetiroDeLote	UoW, Reloj, Almacén	Resultado	Rol y estado → Autorizacion, Lote.retirar
ResurtidoService	generar_alerta / generar_orden_de_compra (CU-05)	ExistenciaBajoMinimo / GenerarOrdenDeCompra	UoW	Alerta u orden	Cantidad sugerida → Inventario; un proveedor por orden → OrdenDeCompra

Responsabilidad dudosa registrada: la verificación de que el rol sea Compras al generar una orden es una regla de permisos que quedó en el servicio, porque aún no existe un Aggregate de usuario/permiso; se anota como deuda técnica.

5. Repository y Unit of Work

Repository (interfaz del dominio)	**Necesidad del dominio**	**Métodos**	**Objeto**
RepositorioDeVentas	Retomar una venta	siguiente_id, agregar, obtener	Venta

	en captura o retenida		
RepositorioDeLotes	Verificar vendibilidad y retirar	obtener(sku, número), de_producto, agregar	Lote
RepositorioDeInventario	Descontar existencia por sucursal	obtener(sucursal, sku), agregar	InventarioDeProducto
RepositorioDeProductos	Precio, controlado, sustitutos, proveedor	obtener(sku), agregar	Producto
RepositorioDeAlertas	Agrupar alertas pendientes por proveedor	agregar, pendientes_de_proveedor	AlertaDeResurtido
RepositorioDeOrdenesDeCompra	Persistir la orden de compra	agregar, obtener	OrdenDeCompra

Las interfaces no exponen SQL. En producción cada una tendría una implementación relacional (un Aggregate = un grupo de tablas, con control de concurrencia optimista); en el código entregado se implementan en memoria para probar el diseño. Todas lanzan EntidadNoEncontrada cuando el Aggregate no existe (mismo contrato, principio L de SOLID).

Unit of Work

- *Inicio:* al entrar al bloque "with" del Application Service (BEGIN).
- *Incluye:* en confirmar_cobro, Venta e InventarioDeProducto (y Lote si se marca caducado); en registrar_retiro, sólo Lote; en generar_orden_de_compra, la orden y las alertas que atiende.
- *Fin:* confirmar() hace commit y después publica los Domain Events.
- *Error:* si el bloque termina sin confirmar o con excepción, se revierte todo (prueba: si otro cajero agota el lote antes del cobro, la venta no queda cobrada). Los rechazos de negocio (receta vencida, lote retirado) se guardan sin tocar el inventario.

6. CQRS básico

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	ConfirmarCobro	venta_id	VentaMostradorService.confirmar_cobro	Resultado (folio/total, rechazo o retenida)	Cambia Venta e Inventario; requiere invariantes y transacción

Command	RegistrarRetiroDeLote	sku, lote, motivo, quién, rol	RetiroDeLoteService.registrar_retiro	Resultado	Cambia el estado del Lote y emite LoteRetirado
Query	ConsultarDisponibilidadEnSucursales	sucursal actual, sku	ConsultarDisponibilidadEnSucursalesHandler	DisponibilidadEnSucursalDTO[]	US-15: sólo cantidades; no cargar Aggregates
Query	ConsultarCaducidadesProximas	sucursal	ConsultarCaducidadesProximasHandler	CaducidadProximaDTO[] (severidad 90/60/30)	US-09/18: modelo de lectura para pantalla y reporte

Los DTO de lectura no reutilizan el modelo de escritura. **No necesitan esta separación:** el resurtido por evento (baja frecuencia) y el catálogo. No se separaron bases de datos.

6. Revisión SOLID

#	Principio	Evidencia	Impacto	Cambio realizado
1	S	Un solo VentaService con venta, autorización, retiro y resurtido mezclaría cuatro razones de cambio	Tocar resurtido obligaría a re-probar la venta	Se dividió en cuatro Application Services

2	S	Límite "10 piezas o $3,000" como literal en el servicio	Regla fuera del dominio y duplicable	LimitesDeVentaControlada (VO); Venta.confirmar_cobro decide
3	O	Elegir lote con "if" por política crecería con cada política	Editar código estable del dominio	Strategy: PoliticaDeOrdenDeLotes + FEFO; otra política = clase nueva
4	L	Repositorios en memoria lanzaban KeyError; uno SQL haría otra cosa	Servicios fallarían distinto según la implementación	Corregido: todos lanzan EntidadNoEncontrada (con prueba)
5	I	Una consulta dependiendo de la Unit of Work vería 6 repositorios que no usa	Dependencias innecesarias	Puerto de lectura LecturaDeInventario (2 métodos). Aceptado: la UoW expone todos los repositorios; mejora futura
6	D	Riesgo de que dominio/aplicación importen SQL o infraestructura	Acoplamiento y pruebas lentas	Puertos en dominio y aplicación; sólo composicion.py conoce lo concreto. Verificado en tests/test_arquitectura.py
7	S/D	Selección de lote dentro de Venta acoplaría tres Aggregates	Venta conocería Lote e Inventario	Domain Service SeleccionDeLote

8. Diagrama de Clases

8.1 Diagrama de clases por capas (dominio, aplicación, infraestructura)

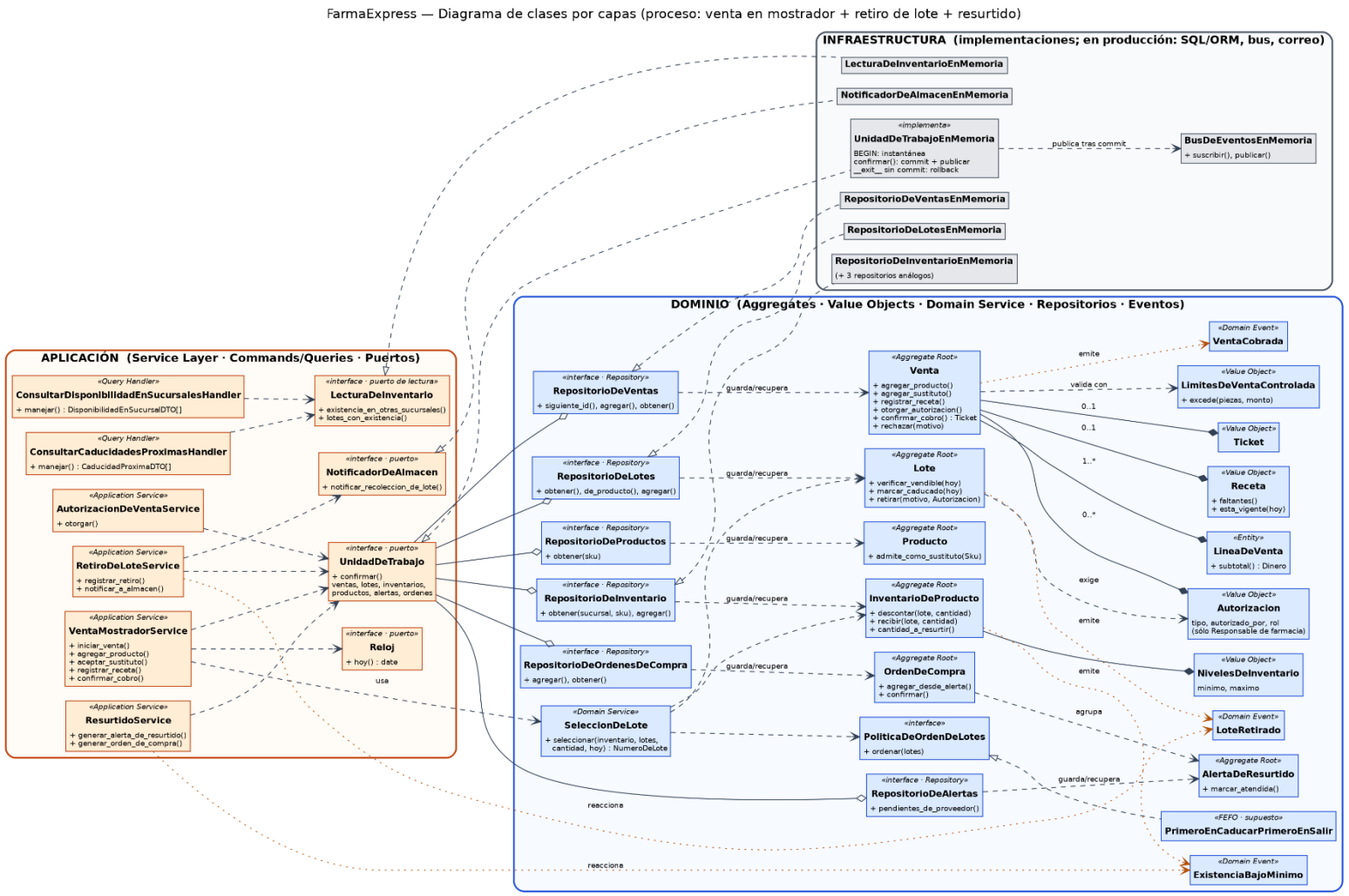


Ilustración 2 Diagrama de clases. Naranja: aplicación · Azul: dominio · Gris: infraestructura. La infraestructura implementa las interfaces; el dominio no depende de nadie.

8.2 Diagrama de secuencia 1 — Flujo principal: cobro de un medicamento controlado (CU-02)

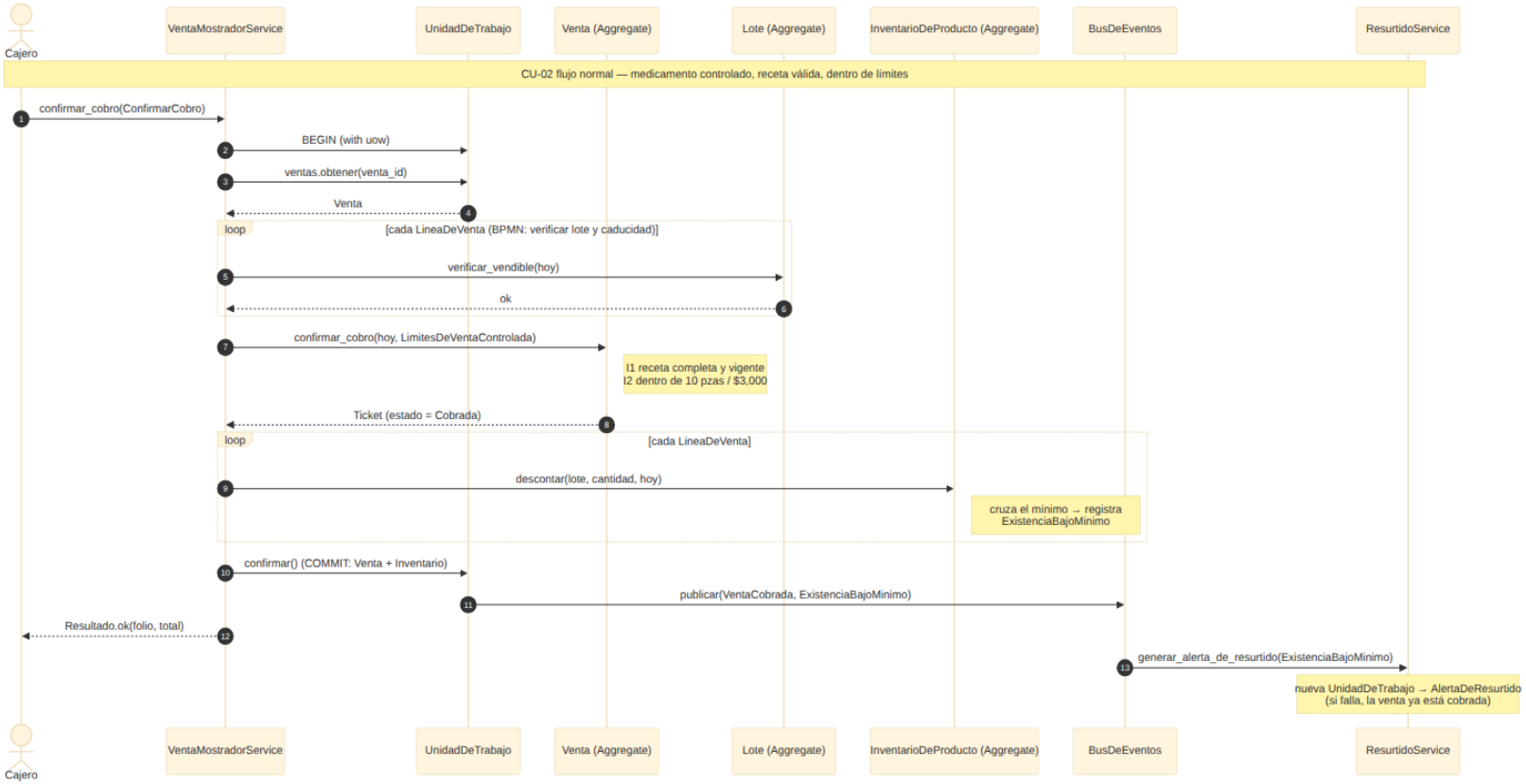
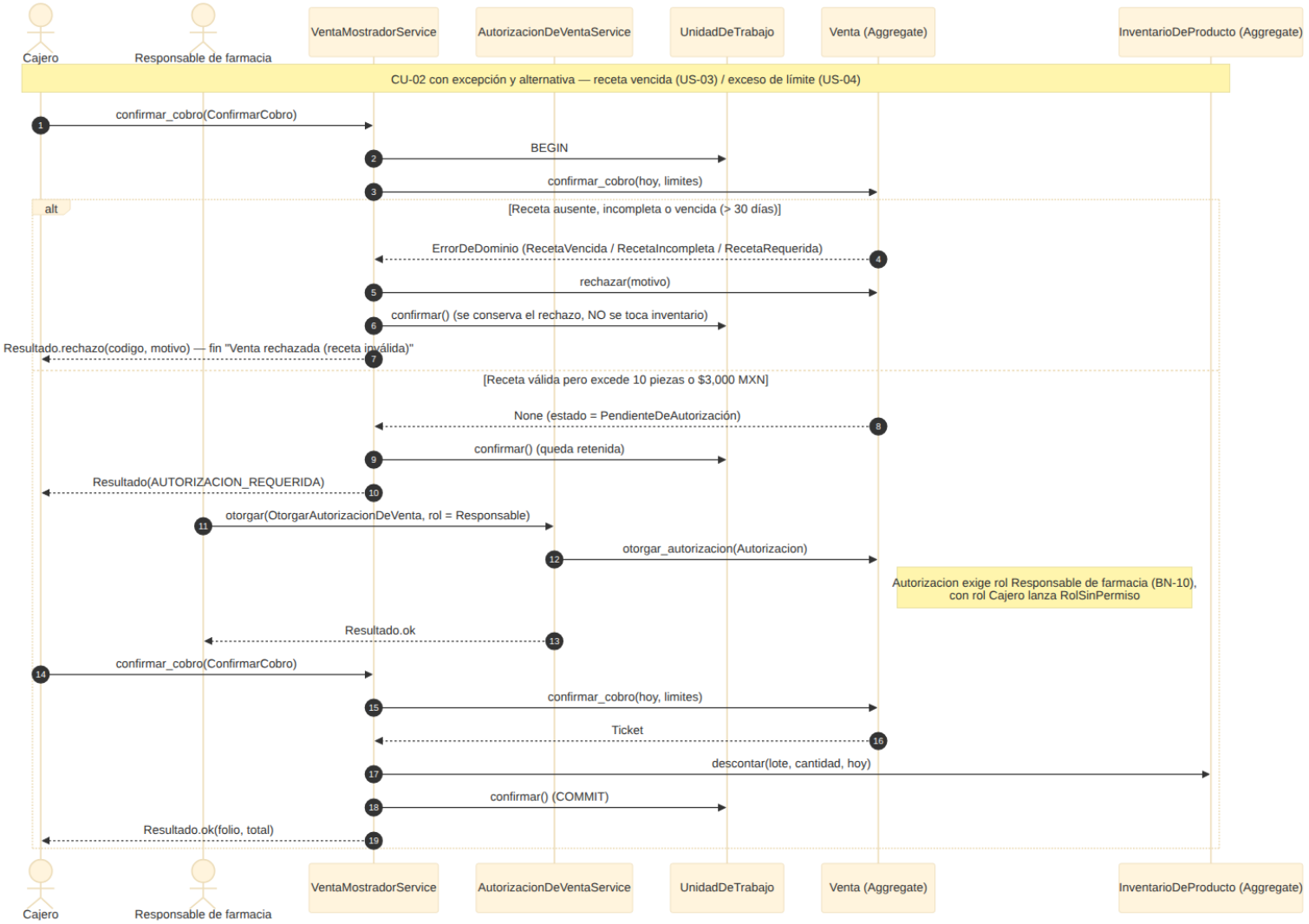


Ilustración 3 Receta válida y dentro de límites: el servicio coordina, Venta valida, el inventario descuenta y los eventos se publican tras el commit.

Se observa que las reglas (receta completa y vigente, límite de piezas y monto) se ejecutan dentro de Venta; el servicio sólo orquesta, abre y confirma la Unit of Work. La alerta de resurtido se genera con una transacción propia, después del commit.



8.3 Diagrama de secuencia 2 — Excepción y alternativa: receta vencida o exceso de límite

Ilustración 4 Rama 1: receta ausente, incompleta o vencida ⇒ rechazo sin tocar inventario. Rama 2: excede 10 piezas o $3,000 ⇒ retenida hasta autorización del responsable.

9. Architecture Decision Records (ADR)

ADR-001 — Domain Model como patrón principal

Estado	Aceptada
Contexto	Restricción regulatoria: ninguna venta de controlado sin receta validada; lote retirado bloqueado en todas las sucursales; 5 gateways exclusivos encadenados en el BPMN.
Alternativas	Transaction Script (reglas duplicadas), Table Module (reglas sin dueño), Domain Model.
Decisión	Domain Model para venta, inventario, retiro y resurtido; Transaction Script sólo en consultas y reportes.
Consecuencias	(+) Reglas probadas sin BD, cambios localizados. (−) Más clases e indirección; los reportes no se benefician del modelo.
Trazabilidad	BN-01, 02, 06, 10 · US-01…08 · BPMN · tests/test_casos_de_uso.py

ADR-002 — Lote como Aggregate global

Estado	Aceptada
Contexto	BN-06/US-08: bloqueo inmediato en todas las sucursales; la existencia es por sucursal y por lote.
Alternativas	Estado en cada sucursal (N escrituras); un Aggregate Producto gigante (contención); Lote global + Inventario por sucursal.

Decisión	Lote (SKU + número) guarda el estado; InventarioDeProducto (sucursal + SKU) guarda cantidades y lo referencia por número.
Consecuencias	(+) Retiro = una sola escritura. (−) Vender requiere leer dos Aggregates y re-verificar el lote antes de cobrar.
Trazabilidad	BN-02, 06 · US-07, 08 · restricción de consistencia del BRD

ADR-003 — Unit of Work de Venta + Inventario; eventos tras commit

Estado	Aceptada
Contexto	Al cobrar deben cambiar la venta y el inventario juntos; la alerta de resurtido no debe poder deshacer una venta válida.
Alternativas	Consistencia eventual del descuento; una sola transacción para todo; UoW de Venta + Inventario con efectos por eventos.
Decisión	Una transacción para Venta e Inventario; los eventos se publican tras el commit y sus manejadores usan su propia UoW.
Consecuencias	(+) Nunca hay venta cobrada sin descuento. (−) Se relaja un-Aggregate-por-transacción; si falla un manejador se pierde la alerta (mitigar con outbox).
Trazabilidad	US-01, 12, 13 · BN-04 · BPMN evento intermedio

ADR-004 — CQRS básico sin bases separadas

Estado	Aceptada

Contexto	Consultas de lectura (US-15, 09, 18) no deben cargar Aggregates ni contaminar el modelo de escritura.
Alternativas	Un solo modelo; CQRS básico; CQRS completo con bases separadas.
Decisión	Commands en Application Services; Queries en Query Handlers con DTO propios y puerto de lectura.
Consecuencias	(+) Consultas simples, responsabilidades claras. (−) Dos caminos de código y DTO adicionales.
Trazabilidad	BN-05, 07, 09 · US-09, 15, 18

ADR-005 — Receta como Value Object; reglas en el dominio

Estado	Aceptada (con supuestos por validar)
Contexto	BN-01/US-02-03 exigen receta completa y vigente; BN-10 restringe quién autoriza. El BPMN y las historias discrepan sobre quién valida (H-02).
Alternativas	Receta como Entidad; validar en el servicio; Value Object con validación en Venta.
Decisión	Receta (VO) con faltantes() y esta_vigente(); Venta aplica I1–I3; Autorizacion sólo se construye con rol Responsable.
Consecuencias	(+) La invariante no puede evadirse; umbrales aislados. (−) Si hay surtido parcial, Receta pasa a Entidad; hay que actualizar BPMN y BRD.
Trazabilidad	BN-01, 10 · US-02, 03, 04, 06 · restricción BRD

10. Hallazgos sobre la Unidad 2

Al cruzar BRD, historias, BPMN y glosario se detectaron los siguientes puntos. No

ID	Hallazgo	Decisión en U3	Estado
H-02	El BPMN asigna la validación de receta al Responsable; BN-01 y US-02/03 piden validación automática	El dominio valida; el Responsable autoriza excepciones	Cambio propuesto al BPMN
H-03	El límite 10 piezas / $3,000 (US-04) no está en el BRD ni en el BPMN	Se modela como LimitesDeVentaControlada	Cambio propuesto
H-04	La autorización para sustituir controlados (US-06) no aparece en el BPMN	Venta.agregar_sustituto exige Autorizacion	Cambio propuesto
H-05	La vigencia de 30 días sólo aparece en US-03; el BRD pide validar el marco legal	Constante aislada Receta.VIGENCIA_EN_DIAS	Supuesto por validar
H-06	No se define de qué lote se descuenta (US-01)	Política intercambiable; FEFO	Supuesto por validar
H-07	"Sucursales cercanas" (US-15) no se define	Se consultan todas las demás	Por validar
H-10	El glosario carece de Venta, Línea de venta, Ticket, Alerta de resurtido, Proveedor; "FarmaExpres" mal escrito	Glosario ampliado	Corregido en U3

11. Validación de la propuesta

Validación	Resultado del equipo
Trazabilidad	Cada componente responde a una necesidad, historia o caso de uso (matriz en docs/05-trazabilidad.md). Pendientes explícitos: US-10, 11, 14, 16, 17, 18.
Lenguaje	Los nombres del código y diagramas coinciden con el glosario; los términos nuevos se agregaron al glosario.
Responsabilidad	Las reglas están en Venta, Lote, Receta, Autorizacion e InventarioDeProducto; los servicios coordinan. Excepción documentada: permiso del rol Compras.
Consistencia	Las secuencias usan las mismas clases del diagrama de clases.
Persistencia	El dominio no importa SQL ni infraestructura; verificado con pruebas automáticas.
Transacción	Límite y manejo de error definidos por la Unit of Work (ADR-003).
IA	Aportes registrados en la bitácora (docs/07-bitacora-ia.md), con errores detectados y corregidos.

Código base y pruebas: el repositorio incluye el código en Python organizado en dominio, aplicación e infraestructura y **21 pruebas automáticas que pasan** (un escenario por caso de uso, rollback transaccional, consultas y reglas de dependencia entre capas).

12.Matriz de Trazabilidad

Funcionalidad / Módulo	Necesidades / Requerimientos	Reglas de Negocio Asociadas	Historias de Usuario (US)	Casos de Uso / Servicios de Aplicación	Componentes / Aggregates Involucrados	Decisiones Arquitectónicas (ADR)
1. Venta de productos sin restricciones	Registrar la venta en mostrador descontando existencia de un lote vendible y emitiendo el ticket.	Existencia suficiente; lote vendible (no retirado ni caducado); descuento; emisión de ticket.	US-01	CU-01 agregar_producto, confirmar_cobro (VentaMostradorService)	Venta, LineaDeVenta, Ticket, Lote, InventarioDeProducto, Producto, SeleccionDeLote (Domain Service)	ADR-001 (Domain Model) ADR-002 (Lote global) ADR-003 (UoW + eventos)
2. Venta de medicamentos controlados	Garantizar que ningún controlado se venda sin receta validada, y retener la venta si excede límites hasta autorización.	BN-01: receta completa y vigente (≤ 30 días). BN-10: solo el Responsable de farmacia autoriza. Límite de 10 piezas / $3,000 requiere autorización.	US-02, US-03, US-04	CU-02 registrar_receta, confirmar_cobro (VentaMostradorService) otorgar (AutorizacionDeVentaService)	Venta, Receta (VO), Autorizacion (VO), LimitesDeVentaControlada (VO), InventarioDeProducto	ADR-001 (Domain Model) ADR-003 (UoW + eventos) ADR-005 (Receta como VO)
3. Productos sustitutos	Ofrecer un sustituto equivalente cuando el producto solicitado no está disponible.	El sustituto debe ser equivalente; en controlados la sustitución exige autorización del Responsable.	US-05, US-06	CU-03 agregar_producto, aceptar_sustituto (VentaMostradorService)	Venta, Producto, Autorizacion (VO), SeleccionDeLote (Domain Service)	ADR-001 (Domain Model) ADR-005 (Receta como VO / autorización)
4. Bloqueo y retiro de lotes	Bloquear de inmediato en todas las sucursales los lotes caducados o retirados y notificar a Almacén.	BN-02: el retiro exige autorización del Responsable. BN-06: el retiro bloquea el lote en todas las sucursales y notifica a Almacén.	US-07, US-08	CU-04 registrar_retiro (RetiroDeLoteService)	Lote (Aggregate global), Autorizacion (VO), evento LoteRetirado, NotificadorDeAlmacen (puerto)	ADR-001 (Domain Model) ADR-002 (Lote como Aggregate global)
5. Resurtido automático	Generar alerta cuando la existencia cruce el punto de	BN-04: existencia < punto de reorden genera alerta; una orden de	US-12, US-13	CU-05 generar_alerta, generar_orden_de_compra	InventarioDeProducto, NivelesDeInventario (VO), AlertaDeResurtido,	ADR-001 (Domain Model) ADR-003 (UoW + eventos

	reorden y emitir la orden de compra al proveedor.	compra por proveedor; no se confirma vacía.		(ResurtidoService)	OrdenDeCompra, evento ExistenciaBajoMinimo	tras commit)
6. Consulta de disponibilidad en sucursales	Consultar existencias del producto en las demás sucursales sin cargar Aggregates.	BN-05, BN-07, BN-09 (según ADR-004).	US-15	ConsultarDisponibilidadEnSucursales (Query con CQRS básico) ConsultarDisponibilidadEnSucursalesHandler	LecturaDeInventario (puerto de lectura), DisponibilidadEnSucursalDTO	ADR-004 (CQRS básico sin bases separadas)
7. Reportes de caducidades	Mostrar y reportar los lotes próximos a caducar con su nivel de severidad.	BN-05, BN-07, BN-09 (según ADR-004).	US-09, US-18	ConsultarCaducidadesProximas (Query con CQRS básico) ConsultarCaducidadesProximasHandler	CaducidadProximaDTO (severidad 90/60/30), LecturaDeInventario (puerto de lectura)	ADR-004 (CQRS básico sin bases separadas) ADR-001 (Transaction Script solo en consultas)

13. Bitácora del uso de la IA

Iteración	Objetivo	Error o supuesto detectado	Corrección
1	Casos de uso, patrón y mapa de dominio a partir de la U2	Supuestos: FEFO, vigencia de 30 días, "sucursales cercanas", Receta como VO	Registrados como hallazgos por validar
2	Diagramas fuente legibles	El diagrama de clases en Mermaid salió ilegible; el mapa tenía flechas cruzadas	Se rehicieron en Graphviz; Mermaid sólo en secuencias
3	Código base y pruebas	Dato de prueba fuera del umbral de 90 días	Se corrigió el dato, no la regla
4	Revisión SOLID	Repositorios lanzaban KeyError (violación de L)	EntidadNoEncontrada y pruebas de arquitectura

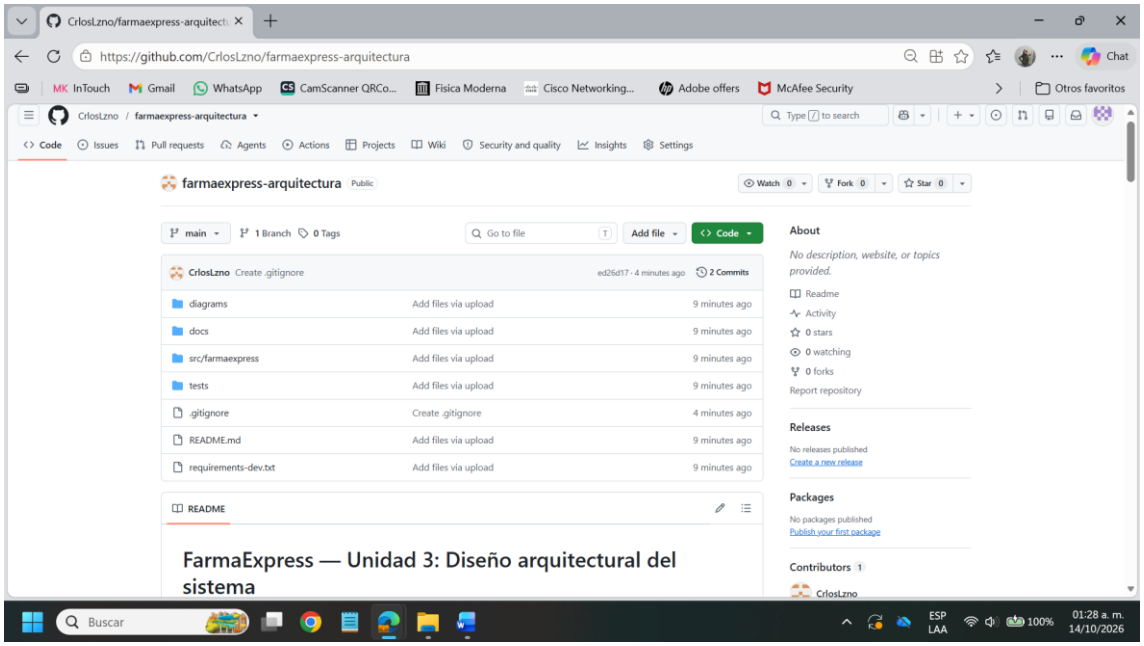
Error o supuesto detectado: Repositorios lanzaban KeyError (violación de L)

Corrección: EntidadNoEncontrada y pruebas de arquitectura

13. Repositorio en GitHub

Todos los entregables se organizaron en un repositorio con una estructura clara.

https://github.com/CrlosLzno/farmaexpress-arquitectura/invitations



Conclusiones

Carlos Patricio Lozano Treviño Decidimos que Domain Model fue la decisión correcta para FarmaExpress porque su riesgo principal es regulatorio: la regla "no se vende un controlado sin receta válida" quedó protegida dentro de Venta y no depende de que cada servicio la recuerde. El costo es una arquitectura con más piezas, además de que separar “Lote de Inventario por sucursal” resolvió la tensión entre bloquear un lote en todas las sucursales al instante (BN-06) y llevar existencias por sucursal; fue la decisión de diseño más difícil de ubicar pero totalmente funcional al final.

Ángel Mauricio Castruita Márquez La Service Así mismo al paso de ir trabajando, distinguimos que la Service Layer debe coordinar, no decidir.

Revisarla con SOLID nos llevó a dividirla en cuatro servicios, a mover los límites al dominio y a encontrar un incumplimiento real del principio L en los repositorios. Por otro lado la Unit of Work que abarca Venta e Inventario evita vender sin existencia real, a cambio de relajar la regla de un Aggregate por transacción y de aceptar el riesgo de perder una alerta si falla un manejador de eventos (mitigable con un outbox).

Arleth García Martínez Revisar a fondo la arquitectura de la Unidad 2 nos reveló inconsistencias (límites de venta y autorización de sustitución ausentes del BPMN, validación de receta atribuida a distintos actores) por consiguiente documentarlas como hallazgos, con las reglas existentes reglas, fue parte esencial del trabajo. Cabe mencionar que la IA aceleró los borradores, pero exigió nuestra revisión como persona física dado que: produjo diagramas ilegibles, un contrato de repositorio inconsistente y supuestos que sólo el negocio puede confirmar y que como equipo somos responsables de validar y defender.